

Project Documentation: Indoor Feedback System

1. Project Overview

The **Indoor Feedback System** is a web-based application designed to monitor, manage, and analyse environmental data from distributed sensor stations. The system supports multi-user interaction with distinct privileges (User vs. Admin), allowing for data collection, visualization, social sharing, and administrative oversight.

1.1 Technology Stack

- **Operating System:** Linux (Debian/Ubuntu based)
 - **Web Server:** Apache HTTP Server
 - **Database:** MySQL / MariaDB
 - **Backend Language:** PHP (Native/Vanilla)
 - **Frontend:** HTML5, CSS3, PHP
-

2. System Architecture & Security

2.1 Authentication & Authorization

- **Session Management:** Uses PHP `$_SESSION` to persist user state across pages.
- **Password Security:** Passwords are hashed using `password_hash()` (Bcrypt) upon registration and verified via `password_verify()` during login.
- **Role-Based Access Control (RBAC):**
 - **Standard Users:** Can manage their own stations, view their own data, create collections, and share with friends.
 - **Administrators:** Have global access to all data, users, and stations. Protected by strict role checks at the top of every Admin PHP file:

PHP

```
$current_role = isset($_SESSION['role']) ? strtolower(trim($_SESSION['role'])) : '';
```

```
if ($current_role !== 'admin') { header("Location: index.php"); exit(); }
```

2.2 Input Handling

- **Sanitization:** All user inputs (`$_POST`, `$_GET`) are passed through `mysqli_real_escape_string` to prevent SQL Injection attacks.
 - **Error Handling:** Custom error reporting (enabled during debugging, suppressed in production) to handle database connection failures gracefully.
-

3. Database Schema (Reconstructed)

The application relies on a relational database with the following key entities:

1. **users:** Stores user credentials and system roles (admin or user).
 2. **station:** Represents physical hardware.
 - *Fields:* serialNumber (PK), name, fk_user_owns.
 3. **measurement:** Time-series data points reported by stations.
 - *Fields:* pk_id, value, time (or timestamp), fk_station_serialNumber.
 4. **collection:** Logical grouping of measurements created by users.
 - *Fields:* pk_id, name, fk_user_owns.
 5. **collection_items** (Junction Table): Links measurements to collections (Many-to-Many relationship).
-

4. Feature Implementation Details

4.1 Global Station Management (Admin)

- **File:** admin_stations.php
- **Functionality:**
 - **Global View:** Lists every station in the system regardless of owner.
 - **Ownership Transfer:** Admins can reassign a station from one user to another via a dropdown menu.
 - **CRUD:** Capabilities to rename stations or permanently delete them.

4.2 Advanced Data Management (Admin)

- **File:** admin_measurements.php
- **Functionality:**
 - **Universal Data Access:** View measurements from all users and all stations.
 - **Dynamic Filtering:** Implemented a robust filter engine that constructs SQL WHERE clauses dynamically based on input:
 - *Date Range:* Converts HTML datetime-local format (T separator) to SQL format (Space separator).
 - *Station ID:* Filters by specific station Serial Numbers.
 - **Bulk Actions:**
 - **Delete:** Permanently remove specific data points.
 - **Add to Collection:** Administrators can inject any measurement into any collection, regardless of ownership.
 - **Schema Auto-Detection:** The system dynamically detects if the time column is named time or timestamp to prevent SQL errors.

4.3 Collection Management

- **File:** admin_collections.php
- **Functionality:**
 - **Global Oversight:** View all collections created by all users.
 - **Renaming:** modify the name of any collection.
 - **Deletion:** Delete collections (and cascade delete their associations).
 - **Conditional Sharing:**
 - Logic was implemented to enforce privacy: *Admins can only see the "Share" button for collections they personally created.*
 - *Logic:* `$is_mine = (strcasecmp($row['fk_user_owns'], $_SESSION['username']) === 0);`

4.4 User Management

- **File:** admin_users.php
- **Functionality:**
 - List all registered users.
 - Force reset passwords.
 - Delete users (and handle the cleanup of their associated stations/data).

5. Technical Challenges & Solutions Implemented

During the development process, several specific technical hurdles were overcome:

5.1 Date Format Conflicts

- **Problem:** HTML5 `<input type="datetime-local">` returns ISO 8601 format (e.g., 2023-10-27T14:30), but MySQL DATETIME expects 2023-10-27 14:30.
- **Solution:** Implemented a pre-processing step in PHP to sanitize the input before the query:

PHP

```
$s = str_replace('T', ' ', $_GET['filter_start']);
```

5.2 Variable Scope in Role Validation

- **Problem:** Early versions of the admin pages failed with "Access Denied" due to inconsistent variable naming (`$user_role` vs. `$current_role`).
- **Solution:** Standardized the security check block across all Admin PHP files to consistently use `$current_role` derived from the active session.

5.3 Empty Data / Debugging Mode

- **Problem:** Admin pages initially showed blank tables, making it unclear if the database was empty or the query failed.
 - **Solution:**
 - Implemented a "Debug Mode" that reports the total row count of the database table at the top of the page.
 - Added explicit error reporting `ini_set('display_errors', 1)` to catch column name mismatches (e.g., Unknown column 'time').
-

6. Current Status

The system is currently functional with a clear separation between **User** (Front-end usage) and **Admin** (Back-end management). The database connectivity is stable, and the "CRUD" (Create, Read, Update, Delete) operations are fully implemented for the core entities (Stations, Data, Collections).